1 Parallel Algorithms: Parallel DFS

Learning Outcomes

1. Explain why a strict depth-first search tree is difficult to construct in parallel.
2. Distinguish a parallel depth-biased work-pool traversal from a true DFS traversal.
3. Describe the divide-and-conquer strategy for constructing a DFS tree using a spine path and hanging components.
4. Use parallel spanning-tree construction, Boruvka rounds, and parallel union–find as building blocks for graph algorithms.
5. Explain pointer jumping as a method for compressing parent pointers and identifying connected components.
6. Construct an Euler tour representation of a tree and reduce depth/path computation to parallel list ranking.
7. Attach off-spine connected components to the correct spine vertices using a deterministic rule.
8. Merge independently computed DFS subtrees into a global DFS tree using subtree sizes and parallel prefix sums.
9. Classify non-tree edges in a merged DFS tree using DFS number intervals.
10. Reason about the gap between parallel graph traversal and a verified DFS tree.

Recap from Previous Lecture

- Parallel algorithms must be analyzed using both *work* (total operations) and *span* or *critical path* (longest dependence chain).
- Good parallel algorithms usually expose independent subproblems, avoid centralized bottlenecks, and replace serial scans with reductions or prefix sums.
- Graph algorithms are especially subtle because local decisions may impose global structural constraints.
- Synchronization primitives such as locks and atomics can make shared-state graph traversal correct, but they do not automatically preserve a desired traversal order.
- Prefix sums, reductions, and pointer jumping are recurring tools for turning global-looking graph operations into parallel primitives.

1.1 The Problem: Why Parallel DFS Is Hard

Depth-first search (DFS) is one of the cleanest serial graph algorithms: start at a root, recursively visit one unvisited neighbour, continue down that path as far as possible, and backtrack only when the current branch is exhausted. This description hides a strong sequential dependency. At any moment, the next recursive call depends on the entire history of earlier choices.

Definition **A DFS tree** is the tree of first-discovery edges produced by a depth-first traversal. For an undirected graph, every non-tree edge in a valid DFS tree connects a vertex to one of its ancestors or descendants in that tree. This ancestor–descendant property is the structural signature that separates

DFS from an arbitrary graph traversal.

The difficulty is not merely that DFS uses a stack. A stack can be shared between processors. The difficulty is that a *specific DFS order* is a serial object. If the serial rule says "always explore the smallest unvisited neighbour first," then the algorithm must know whether the smallest neighbour's entire recursive subtree has finished before it is allowed to explore the next neighbour. That decision creates a long dependence chain.

Pitfall Starting many recursive calls from the same vertex at the same time does not produce a DFS. If a processor explores neighbour B while another explores neighbour C, the resulting discovery pattern may be a useful parallel traversal, but it is not the same as depth-first exploration from B followed by backtracking to C.

1.1.1 Three Levels of Ambition

It is useful to separate three goals:

1. **Parallel search:** find reachable vertices quickly, without requiring a valid DFS tree.
2. **Some valid DFS tree:** construct any tree satisfying the DFS ancestor–descendant property.
3. **A specific DFS order:** reproduce a particular serial DFS order, such as smallest-neighbour-first DFS.

The first goal is achievable with simple shared work pools. The second is much harder and requires structural reasoning about the graph. The third is harder still because it asks a parallel computation to reproduce a serial tie-breaking history.

Key Idea The lecture focuses on the conceptual path from a simple parallel traversal to a divide-and-conquer construction of a DFS tree. The main lesson is not that DFS becomes easy; it is that valid DFS structure must be engineered deliberately.

1.2 Work-Pool Parallel Search

A natural first attempt is to keep a shared container of frontier vertices. Many processors repeatedly remove a vertex, inspect its adjacency list, claim unvisited neighbours, and insert newly claimed vertices back into the container. If the container is a stack, the search is *depth-biased*; if it is a queue, it is breadth-biased.

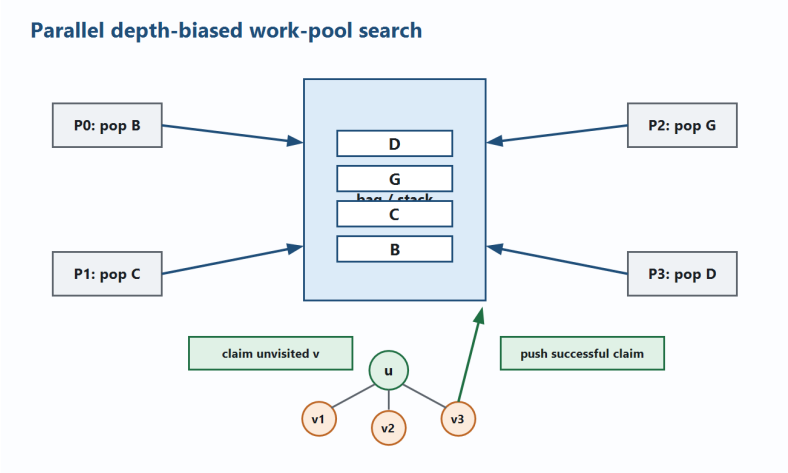

Figure 1.1: *A depth-biased work pool. Processors pop frontier vertices, claim unvisited neighbours, and push successful claims back into the shared bag.*

Program 1.1: *Parallel depth-biased work-pool search*

```
parallel_work_pool_search(Graph G, Vertex r):
    for each vertex v in parallel:
        visited[v] = false
```

```
        parent[v] = NIL

    visited[r] = true
    push(pool, r)

    parallel while pool is not empty:
        u = pop(pool)
        for each neighbour v of u:
            if atomic_compare_and_swap(&visited[v], false,
    true):
                parent[v] = u
                push(pool, v)
```

The atomic compare-and-swap is essential. Two processors may see the same unvisited neighbour v from different endpoints. Only one processor should win the right to mark v, set its parent, and push it into the pool. Without that atomic claim, the algorithm can assign multiple parents to the same vertex.

Definition **Parallel depth-biased search** is a work-pool traversal that uses a stack-like or bag-like frontier to encourage deep exploration. It may be effective as a reachability algorithm, but it is not guaranteed to output a strict DFS tree.

Pitfall The word *stack* is not magic. A shared stack gives a last-in-first-out flavour, but concurrent pushes and pops interleave nondeterministically. The resulting parent tree depends on timing and may violate the DFS ancestor–descendant property.

1.2.1 What This Attempt Gets Right

The work-pool algorithm is still valuable:

- It exposes parallelism across adjacency-list scans.

- It uses a simple correctness rule: each vertex is claimed once.
- It is often good enough for graph reachability, connected-component exploration, and heuristic search.
- It introduces a common pattern: shared frontier plus atomic visitation.

However, it does not solve the lecture's harder question: *How can we obtain a tree with DFS-like structure in parallel?*

1.3 Divide and Conquer for Parallel DFS

The divide-and-conquer idea is to carve out a path that behaves like the main recursive spine of a DFS tree. After removing that path, the remaining vertices break into connected components. Each component hangs from one or more vertices of the spine. These hanging components can then be solved recursively in parallel.

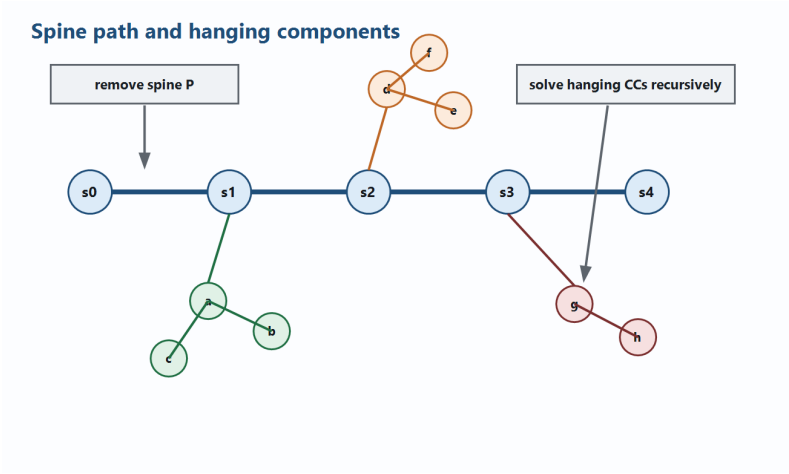

Figure 1.2: *A spine path P and the off-spine connected components that hang from it. Each hanging component becomes an independent recursive subproblem after attachment is fixed.*

Definition **Spine path** means a root-to-leaf path selected from a spanning tree of the graph. It acts as the serial backbone of the DFS tree. The vertices not on the path are partitioned into *hanging components*, which can be processed recursively.

1.3.1 High-Level Algorithm

The lecture's divide-and-conquer outline is:

1. Compute a spanning tree S of the graph in parallel.
2. Extract a root-to-leaf path P from S. This path is the spine.
3. Remove or contract all vertices of P from the graph.
4. Find the connected components hanging off P.
5. Assign each component to an attachment vertex on the spine.
6. Recursively compute DFS trees of the components in parallel.
7. Merge the recursive trees into one global DFS tree.
8. Verify the DFS property if the construction needs an explicit certification step.

Key Idea The algorithm moves the serial part into a path and tries to make the remaining work independent. This is the same divide-and-conquer instinct used in many parallel algorithms: isolate a separator, solve the pieces, and merge carefully.

1.3.2 Why a Spanning Tree Helps

A spanning tree gives a connected skeleton of the graph. From that skeleton we can select a root-to-leaf path. The path is not arbitrary: it provides a chain along which DFS numbers can be ordered before placing the hanging recursive blocks.

Note For DFS construction, the spanning tree is a structural tool. If the graph is weighted and we compute a minimum spanning tree, that tree is certainly a spanning tree. If the graph is unweighted, the weights may be artificial tie-breakers. The DFS procedure needs connectivity and a path skeleton, not the optimality of a minimum spanning tree.

1.4 Parallel Spanning Tree Construction

The slides describe the spanning-tree step using parallel Kruskal-style ideas and Boruvka rounds. The common goal is to connect components by repeatedly selecting safe edges in parallel.

1.4.1 Parallel Sorting of Edges

Kruskal's algorithm begins by sorting edges by weight. Sorting is a well-understood parallel primitive. One simple strategy is:

1. Split the edge list into p chunks.
2. Sort each chunk locally in parallel.
3. Merge the sorted chunks using a parallel merge tree.

For example, the edge-weight list

$$[38, 27, 43, 3, 9, 82, 10, 19]$$

may be split across two processors:

$$T_1 : [38, 27, 43, 3] \to [3, 27, 38, 43],$$
$$T_2 : [9, 82, 10, 19] \to [9, 10, 19, 82].$$

The two sorted sublists are then merged.

Definition **Parallel merge** merges two sorted arrays by choosing a splitter in one array, binary-searching its position in the other array, and recursively merging the left and right halves in parallel. The split creates independent subproblems while preserving sorted order.

Program 1.2: *Recursive parallel merge idea*

```
parallel_merge(A, B):
    if |A| + |B| is small:
        merge serially
        return

    ensure |A| >= |B|
    x = median element of A
    k = lower_bound(B, x)

    // Elements before A[mid] and before B[k] belong to the
     left output.
    spawn parallel_merge(A_left,  B_left)
    spawn parallel_merge(A_right, B_right)
    sync
```

The exact complexity depends on the merge implementation and machine model. The important structural point is that binary search finds a correct split, after which the two halves can proceed independently. Across many chunks, a tree of merge stages reduces p sorted lists to one sorted list.

1.4.2 Boruvka Rounds

After sorting, the lecture moves to a Boruvka-style component contraction. Initially, every vertex is its own component. In each round:

1. Every component finds its minimum-weight outgoing edge.

2. Selected edges are de-duplicated because two components may choose the same edge.
3. The chosen edges are added to the forest.
4. Components connected by chosen edges are contracted into super-vertices.

Definition **Boruvka round** is a parallel phase in which each current component chooses one cheapest outgoing edge. All such choices can be made concurrently because each component's local minimum can be computed independently.

Deduplication matters. If component C_1 selects edge (A, C) and component C_2 also selects the same edge (A, C), the edge should be inserted only once. A parallel scan or sort-by-edge-id can compact the selected list into unique edges.

Key Idea Boruvka rounds are parallel because the algorithm asks many components the same local question at the same time: "What is your cheapest outgoing edge?" The contraction step then reduces the number of components, often by a constant factor per round.

Program 1.3: *Borůvka spanning-tree algorithm*

```
boruvka_spanning_tree(Graph G):
    for each vertex v in parallel:
        component[v] = v          // single-vertex
    components
    T = empty edge set

    while more than one component exists:
        // Each component finds its cheapest outgoing edge.
        for each component C in parallel:
```

```
            cheapest[C] = NIL
            for each edge (u, v, w) with Find(u)==C, Find(v
)!=C:
                if cheapest[C]==NIL or w < weight(cheapest[
C]):
                    cheapest[C] = (u, v, w)

        // Deduplicate: two components may pick the same
edge.
        selected = deduplicate({ cheapest[C] : cheapest[C]
!= NIL })
        T = T union selected

        // Contract: merge endpoints of each selected edge.
        for each edge (u, v) in selected in parallel:
            union(component, u, v)
        pointer_jumping(component)    // flatten to roots

    return T
```

Note Each Borůvka round merges at least two components per selected edge, so the number of components drops by at least half per round. Starting from n singleton components, at most $O(\log n)$ rounds suffice. Combined with $O(m)$ work per round, total work is $O(m \log n)$.

1.4.3 Parallel Union–Find with Pointer Jumping

Once the algorithm has selected de-duplicated edges such as

$$(A, C), \quad (B, E), \quad (B, D), \quad (D, F),$$

it must update the component structure. This is the job of union–find. The parallel version uses parent pointers and pointer jumping

to compress paths.

Definition **Pointer jumping** repeatedly replaces a node's parent by its grandparent:

$$parent[v] \leftarrow parent[parent[v]].$$

After $O(\log n)$ synchronous rounds on a tree-like parent structure, long paths shrink rapidly toward their roots.

Program 1.4: *Pointer jumping for component roots*

```
parallel_find_roots(parent):
    changed = true
    while changed:
        changed = false
        for each vertex v in parallel:
            old = parent[v]
            parent[v] = parent[parent[v]]
            if parent[v] != old:
                changed = true
```

For an edge (u, v), the edge crosses components exactly when

$$Find(u) \neq Find(v).$$

This gives a constant-time test after the roots have been computed. Each processor can inspect one edge, record whether it is cross-component, and contribute its weight to a parallel minimum reduction.

Pitfall Parallel union–find must be designed carefully. If many processors update parent pointers without a consistent rule, they can create cycles in the parent forest. Practical imple-

mentations use deterministic linking rules, atomic compare-and-swap, or phased algorithms to keep the structure well formed.

1.5 Extracting a Root-to-Leaf Path

After computing a spanning tree S, the divide-and-conquer DFS algorithm needs a root-to-leaf path. The lecture presents an Euler tour approach followed by parallel list ranking.

1.5.1 Euler Tour of a Tree

Take every undirected tree edge $\{u, v\}$ and replace it by two directed arcs:

$$u \to v \qquad \text{and} \qquad v \to u.$$

The Euler tour is represented by a `next` pointer on each directed arc. The pointer tells us which arc is visited next in the tree traversal.

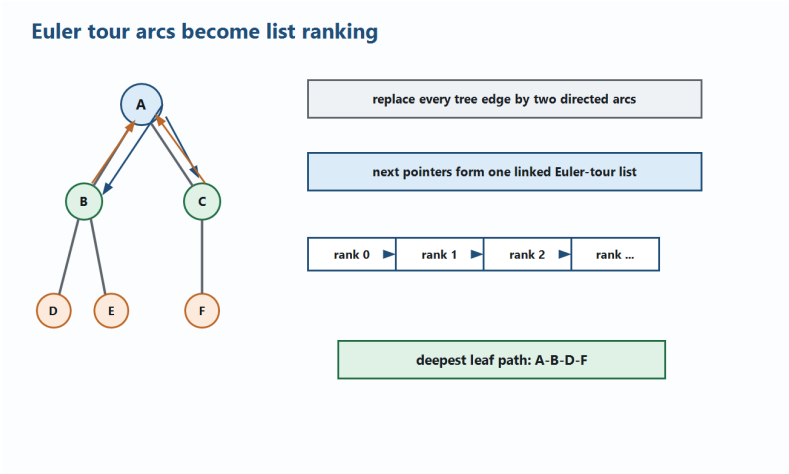

Figure 1.3: *Euler tour construction. Directed tree arcs are linked by next pointers, converting tree traversal into a list-ranking problem.*

The next-pointer rules are:

1. For an entering arc $u \to v$, if v has children, the next arc goes from v to its first child. If v is a leaf, the next arc is $v \to u$, immediately returning to the parent.
2. For a returning arc $v \to u$, the next arc out of u goes to the next child after v. If v was the last child of u, the next arc returns from u to $parent(u)$.
3. At the root, after the last child has returned, the tour terminates or wraps to a sentinel.

Key Idea The point of the Euler tour representation is that every arc's successor can be computed locally from parent/child ordering. Once the successor pointers exist, the tree problem becomes a linked-list problem.

1.5.2 Parallel List Ranking

List ranking assigns to every item in a linked list its distance from the beginning of the list. With Euler-tour arcs, those ranks encode enough information to recover depths and root-to-leaf paths.

Definition **List ranking** computes the rank of every node in a linked list in parallel. Pointer jumping is again the central trick: each item skips over its successor while accumulating the skipped distance.

Program 1.5: *List ranking by pointer jumping*

```
parallel_list_rank(next):
    for each list item x in parallel:
        rank[x] = 1

    while next[x] is not NIL for some x:
        for each item x in parallel:
            if next[x] != NIL:
```

```
                rank[x] = rank[x] + rank[next[x]]
                next[x] = next[next[x]]
```

Once depths are known, the algorithm can reconstruct paths from leaves to the root in parallel. The deepest leaf gives a natural root-to-leaf path. In the example shown in the lecture, the selected path is

$$A \to B \to D \to F.$$

This path becomes the spine for the next divide-and-conquer step.

1.6 Finding Hanging Components

Let P be the selected spine path. Remove the vertices of P from the graph. The remaining vertices form one or more connected components. Each of these components is a hanging subproblem.

1.6.1 Off-Spine Parent Initialization

The lecture describes a parent-pointer method. For every off-spine vertex v, initialize:

$$off_parent[v] = \begin{cases} parent[v], & \text{if } parent[v] \text{ is also off-spine,} \\ v, & \text{otherwise.} \end{cases}$$

Then repeatedly apply pointer jumping:

$$off_parent[v] \leftarrow off_parent[off_parent[v]].$$

After enough rounds, vertices in the same off-spine tree component point to the same representative.

Program 1.6: *Off-spine component discovery*

```
find_hanging_components(TreeParent parent, Path P):
    for each vertex v not in P in parallel:
```

```
        if parent[v] not in P:
            off_parent[v] = parent[v]
        else:
            off_parent[v] = v

    repeat O(log n) pointer-jumping rounds:
        for each vertex v not in P in parallel:
            off_parent[v] = off_parent[off_parent[v]]
```

Note If the original graph has non-tree edges among off-spine vertices, the full connected-component computation must account for those edges as well. The same union–find idea applies: union all off-spine edges whose endpoints are both outside P, then pointer-jump to representatives.

1.7 Attaching Components to the Spine

A hanging component may have edges to multiple vertices on the spine. To obtain a deterministic DFS-like structure, the lecture uses a simple rule: attach the component to the deepest spine vertex it touches.

Let the spine be ordered as

$$P = (s_0, s_1, \ldots, s_k),$$

where s_0 is closest to the root and s_k is deepest. For a component C_i, inspect all edges (u, s_j) where $u \in C_i$ and $s_j \in P$. Compute

$$j^\star = \max\{j : \exists u \in C_i \text{ such that } (u, s_j) \in E\}, \qquad \mathit{attach}(C_i) = s_{j^\star}.$$

Program 1.7: *Deepest-spine attachment rule*

```
attach_components_to_spine(components C, path P):
```

```
for each component C_i in parallel:
    best_index = -1
    best_entry = NIL

    for each edge (u, s_j) with u in C_i and s_j in P:
        if j > best_index:
            best_index = j
            best_entry = u

    attach_vertex[C_i] = s_best_index
    recursive_root[C_i] = best_entry
```

Key Idea Deepest attachment prevents a component from being pulled upward when it also touches a lower spine vertex. This is important because DFS ordering should finish deeper opportunities before returning to shallower alternatives.

1.7.1 Recursive Calls

After attachment, each component becomes an independent recursive graph problem. If C_i attaches to spine vertex s_j, then one entry vertex inside C_i is chosen as the recursive root. The lecture mentions deterministic choices such as:

- the first edge in the adjacency-list order,
- the smallest endpoint label,
- or another fixed tie-breaking rule.

The recursive calls can run in parallel because the components are vertex-disjoint.

Pitfall The root choice must be deterministic if the algorithm is expected to be reproducible. Otherwise two executions may produce different valid trees, which is often acceptable for correctness but inconvenient for debugging and testing.

1.8 Merging Recursive DFS Trees

After recursively computing DFS trees for the hanging components, we must merge everything into one global DFS tree. This is where the algorithm must be careful: a collection of locally valid DFS trees is not automatically a globally valid DFS tree.

1.8.1 Merge Invariants

The lecture states three invariants for the merge:

1. **Attachment edge:** the edge from the spine vertex to the component root becomes a tree edge.
2. **Preorder respect:** the spine vertex is numbered before all vertices in its hanging component.
3. **Non-tree edge classification:** every non-tree edge is classified using the DFS intervals of its endpoints.

Definition **DFS interval** for a vertex u is the range

$$[dfs[u],\ dfs[u] + size[u] - 1],$$

where $dfs[u]$ is the preorder number of u, and $size[u]$ is the size of u's DFS subtree. A vertex v is a descendant of u exactly when $dfs[v]$ lies inside u's interval.

1.8.2 Computing Global DFS Numbers

Suppose spine vertex s_j has a hanging component block of size $block_size[s_j]$. The global DFS number of spine vertex s_k is:

$$dfs[s_k] = k + \sum_{j<k} block_size[s_j],$$

assuming each earlier spine vertex contributes one slot for itself plus its attached block. This is exactly the kind of computation prefix sums are meant for.

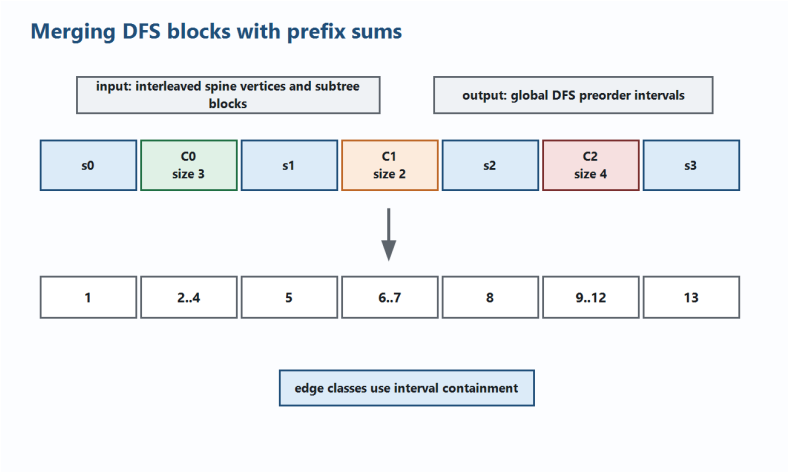

Figure 1.4: *Merging by prefix sums. Spine vertices and component blocks form an interleaved array; prefix sums assign each block its global DFS interval.*

Program 1.8: *Prefix-sum merge numbering*

```
merge_numbering(path P, attached_blocks B):
    // Interleave spine vertices and their component blocks.

    A = []
    for j = 0 to |P|-1:
        append A, item(type = SPINE, size = 1, vertex = P[j
    ])
```

```
        if B[j] exists:
            append A, item(type = BLOCK, size = |B[j]|,
component = B[j])

start = parallel_prefix_sum(A.size)

for each item A[t] in parallel:
    if A[t].type == SPINE:
        dfs[A[t].vertex] = start[t]
    else:
        offset all local DFS numbers in A[t].component
by start[t]
```

Key Idea The prefix sum is the merge glue. It assigns disjoint global number ranges to subtrees without serially walking every component in spine order.

1.8.3 Classifying Non-Tree Edges

After global DFS numbers and subtree sizes are known, a non-tree edge (u, v) can be classified by interval containment:

- If $dfs[u] < dfs[v]$ and

$$dfs[v] \leq dfs[u] + size[u] - 1,$$

  then v lies in u's subtree. The edge is a forward edge from ancestor to descendant.
- If $dfs[v] < dfs[u]$ and

$$dfs[u] \leq dfs[v] + size[v] - 1,$$

  then u lies in v's subtree. The edge is a back edge from descendant to ancestor.

- Otherwise the edge is a cross edge.

For undirected DFS, cross edges are a warning sign: a valid DFS tree of an undirected graph should not have cross edges between unrelated subtrees. Therefore classification can also serve as a verification step.

Pitfall Forward, back, and cross edge terminology is standard for directed DFS. For undirected graphs, every non-tree edge in a correct DFS tree should connect an ancestor and a descendant. So in the undirected setting, a cross edge usually indicates that the proposed merge is not a valid DFS tree.

Note For *directed* graphs the situation differs. Cross edges between unrelated subtrees are perfectly legal in a directed DFS, and forward edges (ancestor to proper descendant, skipping intermediate tree edges) also appear. The interval test still works: a cross edge (u, v) satisfies $dfs[v] < dfs[u]$ with u outside v's subtree interval, while a forward edge has $dfs[u] < dfs[v]$ with v inside u's interval. The absence-of-cross-edges rule applies only to undirected DFS.

1.9 Worked Example

Example Consider the graph G with vertices $\{1, 2, 3, 4, 5\}$ and edges 1–2, 2–3, 2–4, 1–5 (spanning tree S) plus the non-tree edge 3–5.

Step 1 — Spanning tree. Parallel spanning-tree construction produces $S = \{1\text{–}2,\ 2\text{–}3,\ 2\text{–}4,\ 1\text{–}5\}$.

Step 2 — Spine selection. Euler tour and list ranking

compute depths: $d(1) = 0$, $d(2) = 1$, $d(3) = 2$, $d(4) = 2$, $d(5) = 1$. The deepest leaf is 3 (or 4; break ties consistently). Select spine $P = (s_0{=}1,\ s_1{=}2,\ s_2{=}3)$.

Step 3 — Hanging components. Remove $\{1, 2, 3\}$. Two isolated components remain:

- $C_1 = \{4\}$: touches spine only at $s_1{=}2$. Attach to s_1.
- $C_2 = \{5\}$: touches spine at $s_0{=}1$ (via edge 1–5) and $s_2{=}3$ (via edge 3–5). Deepest index is 2, so attach to $s_2{=}3$.

Step 4 — Recursive DFS. Both components are single vertices; their local DFS numbers are trivially 1.

Step 5 — Merge via prefix sums. Interleaved layout: s_0, $\text{block}(s_0){=}\emptyset$, s_1, $\text{block}(s_1){=}\{4\}$, s_2, $\text{block}(s_2){=}\{5\}$. Sizes: $[1, 0, 1, 1, 1, 1]$. Exclusive prefix sums give start offsets $[0, 1, 1, 2, 3, 4]$. Adding 1 for one-based numbering:

$$dfs[1] = 1, \quad dfs[2] = 2, \quad dfs[4] = 3, \quad dfs[3] = 4, \quad dfs[5] = 5.$$

Step 6 — Edge classification. The global DFS tree edges are 1–2, 2–3, 2–4, 3–5. The remaining edge 1–5 is non-tree. Subtree of vertex 1: interval $[1, 5]$. Since $dfs[5] = 5 \in [1, 5]$, vertex 5 is a descendant of 1. Edge 1–5 is a *forward edge* (ancestor to descendant) — valid for an undirected DFS tree.

1.10 Putting the Pieces Together

The full algorithm is best understood as a pipeline of parallel primitives:

1. Parallel sort/selection builds a spanning tree skeleton.
2. Euler tour plus list ranking extracts a deep root-to-leaf path.
3. Pointer jumping and union–find identify off-spine components.

4. A deepest-attachment rule assigns each component to a spine vertex.
5. Recursive calls solve independent components.
6. Prefix sums merge local DFS number ranges into a global order.
7. Interval tests verify or classify non-tree edges.

Program 1.9: *Divide-and-conquer parallel DFS skeleton*

```
parallel_DFS(Graph G, Vertex r):
    if G is small:
        return serial_DFS(G, r)

    S = parallel_spanning_tree(G)
    P = deepest_root_to_leaf_path(S, r)

    components = connected_components(G without vertices of
     P)
    attachments = deepest_spine_attachment(components, P)

    for each component C_i in parallel:
        T_i = parallel_DFS(C_i, root_i)

    T = merge_spine_and_recursive_trees(P, T_i, attachments
    )
    verify_or_classify_non_tree_edges(G, T)
    return T
```

Key Idea This algorithm is not one trick. It is a composition of standard parallel primitives: sorting, reduction, scan, pointer jumping, list ranking, connected components, recursion, and verification. That is why parallel graph algorithms often look more elaborate than their serial counterparts.

1.10.1 Where Parallelism Appears

- Edge sorting and merging use parallel divide-and-conquer.
- Component minimum-edge selection uses parallel reductions.
- Deduplication uses sorting, hashing, or scan-style compaction.
- Union–find uses parallel pointer updates with path compression.
- Euler-tour next pointers are computed independently for arcs.
- List ranking uses pointer jumping.
- Hanging components are solved recursively in parallel.
- Merge numbering uses prefix sums.

1.10.2 Where Sequential Dependence Remains

DFS remains difficult because the spine is still a chain. The algorithm tries to make the chain small enough, or the hanging work large enough, that parallelism is still available. In worst-case graphs that are essentially paths, DFS has little useful parallelism because the graph itself exposes a long dependence chain.

Note A good mental model: parallel DFS does not remove the depth-first dependency. It isolates that dependency into a spine and then parallelizes the subproblems hanging from the spine.

1.10.3 Complexity View

Let $m = |E|$, $n = |V|$, and p be the number of processors. The table below summarises the key steps under a CREW PRAM model.

Step	**Work**	**Span**
Parallel edge sort	$O(m \log m)$	$O(\log^2 m)$
Borůvka spanning tree	$O(m \log n)$	$O(\log^2 n)$
Euler tour construction	$O(n)$	$O(1)$
List ranking	$O(n)$	$O(\log n)$
Off-spine component label	$O(n)$	$O(\log n)$
Deepest attachment scan	$O(m)$	$O(\log m)$
Merge via prefix sums	$O(n)$	$O(\log n)$
Edge classification	$O(m)$	$O(1)$
Total	$O(m \log n)$	$O(\log^2 n)$

Note The $O(\log^2 n)$ span arises from $O(\log n)$ Borůvka rounds each requiring an $O(\log n)$-span sorting or reduction step. On practical shared-memory machines the dominant cost is edge sorting; pointer jumping converges quickly in practice even when the worst-case bound is $O(\log n)$ rounds.

Pitfall Do not judge a parallel graph algorithm only by the number of processors it uses in one step. The critical question is whether the algorithm repeatedly creates balanced independent work, or whether it keeps producing one large serial subproblem.

1.10.4 Key Takeaways

- Strict DFS is hard to parallelize because depth-first order creates a long dependency chain.
- A work-pool traversal can find reachable vertices in parallel, but it does not guarantee a valid DFS tree.

- The divide-and-conquer strategy chooses a spine path, removes it, and recursively solves hanging components.
- Parallel spanning-tree construction uses sorting, Boruvka-style edge selection, deduplication, and union–find.
- Pointer jumping is a unifying primitive for parent compression, connected components, and list ranking.
- Euler tours convert tree traversal into a linked-list ranking problem.
- Deepest-spine attachment gives a deterministic rule for placing hanging components.
- Prefix sums assign global DFS number ranges during merge.
- DFS interval containment classifies non-tree edges and helps verify correctness.

1.10.5 Exercises

Exercises

Exercise 1.1. Why does exploring all neighbours of a vertex in parallel fail to reproduce serial DFS?

Answer: Serial DFS must finish the complete recursive subtree of one neighbour before moving to the next neighbour. If two neighbours are explored simultaneously, their discoveries interleave, so the parent tree may not satisfy the DFS ancestor–descendant property.

Exercise 1.2. In Program 1.1, why is an atomic compare-and-swap needed for `visited[v]`?

Answer: Two processors may discover the same vertex at the same time. The atomic operation ensures that exactly one processor changes `visited[v]` from false to true and therefore exactly one processor assigns `parent[v]`.

Exercise 1.3. Explain how pointer jumping reduces the height

of a parent-pointer tree.

Answer: Each round replaces a vertex's parent by its grandparent. A path of length L is roughly halved each round, so after $O(\log L)$ rounds, vertices point close to or directly at the root.

Exercise 1.4. Let a spine be s_0, s_1, s_2, and suppose the attached component block sizes are $2, 0, 3$. What global DFS numbers are assigned to the spine vertices if numbering starts at 1?

Answer: The order is s_0, block of size 2, s_1, s_2, block of size 3. Thus $dfs[s_0] = 1$, $dfs[s_1] = 4$, and $dfs[s_2] = 5$.

Exercise 1.5. Given $dfs[u] = 4$, $size[u] = 6$, $dfs[v] = 8$, and $size[v] = 2$, classify edge (u, v).

Answer: The interval of u is $[4, 9]$, and $dfs[v] = 8$ lies inside it. Thus v is a descendant of u, so the edge is ancestor-to-descendant from u's perspective.

Exercise 1.6. In the Borůvka algorithm, components C_1 and C_2 both select edge (A, B) as their cheapest outgoing edge. What goes wrong without deduplication, and how does deduplication fix it?

Answer: Without deduplication, (A, B) is inserted twice into the spanning forest. This either creates a multi-edge (if insertion is unconditional) or, when combined with union–find, may trigger an extra union call on already-merged components—breaking the invariant that selected edges connect distinct components. Deduplication removes duplicate edge references before insertion, so each edge is added at most once.

Exercise 1.7. Construct the Euler tour next pointers for the path 1–2–3 rooted at 1. List all directed arcs and their successors.

Answer: The four directed arcs are $1 \to 2$, $2 \to 3$, $3 \to 2$, $2 \to 1$. Following the next-pointer rules: next($1 \to 2$) = $2 \to 3$ (enter 2, first child is 3); next($2 \to 3$) = $3 \to 2$ (3 is a leaf, return immediately); next($3 \to 2$) = $2 \to 1$ (3 was the last child of 2, return to parent 1); next($2 \to 1$) = NIL (1 is the root, tour ends).

Exercise 1.8. Suppose the input graph is itself a path $v_1 – v_2 – \cdots – v_n$. What spine does the algorithm choose, and how much parallel speedup does the divide-and-conquer achieve?

Answer: The only spanning tree is the path itself. The deepest root-to-leaf path is the entire graph, so the spine $P = (v_1, v_2, \ldots, v_n)$ covers all vertices. No hanging components remain after removing the spine, so there are no recursive subproblems to run in parallel. The divide-and-conquer achieves no speedup beyond the single prefix-sum step that assigns DFS numbers to the chain. This is the worst case: a path graph exposes a dependence chain of length n, leaving no exploitable parallelism in the decomposition.

References

This chapter is based on Lecture 26, *Parallel Algorithms*, from `lec26-PA.pdf`. For background on parallel graph algorithms, spanning trees, prefix sums, and parallel performance models, see standard parallel-programming references such as Pacheco's *An Introduction to Parallel Programming*.

Contributions

This chapter transcription and expansion was prepared from the Lecture 26 slide deck. The diagrams in Figures 1.1, 1.2, 1.3, and

1.4 were redrawn as manuscript figures to illustrate the algorithms without relying on raw slide screenshots.

Namit Jain (2023CS10483)

Lecture 26 transcription, chapter structuring, explanations, pseudocode, exercises, and manuscript figures.

Apurva Samota (2023CS10585)

Complexity analysis table, Borůvka pseudocode, worked example, directed-graph DFS note, and additional exercises.